

DAA ASSIGNMENT#02

Saturday, 20 September 2025 6:14 PM

NABIRA KHAN 2314-0914

QUESTION#01

PSEUDOCODE:

```
function Partition(A, lo, hi):  
    pivot = A[hi] // last element as pivot  
    i = lo-1  
    for j = lo to hi-1:  
        if A[j] >= pivot: // for descending  
            i = i+1  
            swap A[i], A[j]  
    swap A[i+1], A[hi]  
    return i+1
```

```
function QuickSort(A, lo, hi):  
    if lo < hi:  
        p = Partition(A, lo, hi)  
        QuickSort(A, lo, p-1)  
        QuickSort(A, p+1, hi)
```

TIME COMPLEXITY:

Best Case $\rightarrow T(n) = 2T(\frac{n}{2}) + O(n)$
 $a=2, b=2, d=1$

$$T(n) = O(n \log n)$$

Avg Case $\rightarrow T(n) = O(n \log n)$

Worst Case $\rightarrow$ already sorted + last element

$$T(n) = T(n-1) + O(n)$$

$$T(n) = O(n^2)$$

QUESTION#02

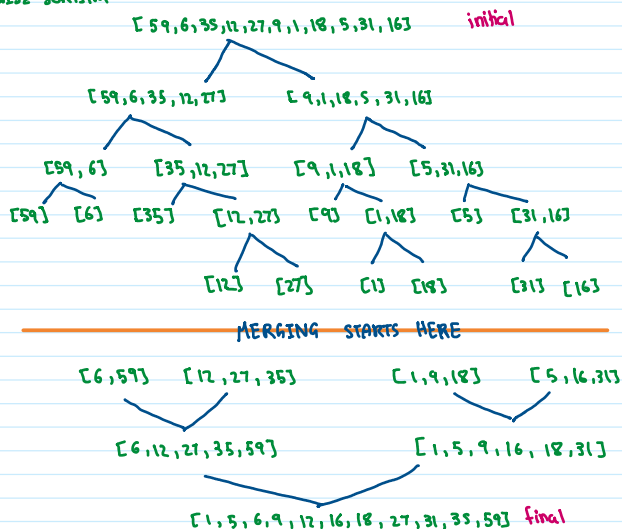
Unsorted List = [59, 6, 35, 12, 27, 9, 1, 18, 5, 31, 16]

① PSEUDOCODE:

```
function Merge(L, R):  
    result = empty list  
    i = 0, j = 0  
    while i < len(L) and j < len(R):  
        if L[i] <= R[j]:  
            append L[i]; i = i+1  
        else:  
            append R[j]; j = j+1  
    append remaining elements from L or R  
    return result
```

```
function MergeSort(A):  
    if len(A) <= 1:  
        return A  
    mid = floor(len(A)/2)  
    L = MergeSort(A[0..mid-1])  
    R = MergeSort(A[mid..end])  
    return Merge(L, R)
```

② STEP-WISE SORTING



③ MERGE SORT WITH 4 PARTS

recurrence relation $\rightarrow T(n) = 4T(\frac{n}{4}) + O(n)$ cost to merge 4 sorted parts

asymptotic running time $\rightarrow a=4, b=4, d=1 \quad O(n)$

$$T(n) = O(n \log n)$$

QUESTION #03

a) PSEUDOCODE

```

function MatMulNaive(A, B, n):
    initialize C as n x n zero matrix
    for i = 0 to n-1:
        for j = 0 to n-1:
            sum = 0
            for k = 0 to n-1:
                sum = sum + A[i][k] * B[k][j]
            C[i][j] = sum
    return C

```

TIME COMPLEXITY

loop for i $\rightarrow O(n)$

loop for j $\rightarrow O(n)$

loop for k $\rightarrow O(n)$

$T(n) = O(n^3)$

for $n=4$, number of scalar multiplications = $4^3 = 64$

b) ALGORITHM

split 4×4 into 2×2 matrices

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

then Strassen defines 7 products (each is a multiplication of 2×2 matrices after linear comb)

$$M_1 = (A_{11} + A_{22}) * (B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22}) * B_{11}$$

$$M_3 = A_{11} * (B_{12} - B_{22})$$

$$M_4 = A_{22} * (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12}) * B_{22}$$

$$M_6 = (A_{21} - A_{11}) * (B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22}) * (B_{21} + B_{22})$$

then recombine to get 4 blocks of C

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

each M is computed by recursive matrix multiply on matrices of size $n/2 \rightarrow$ cost $O(n^{2.81})$
 for 4×4 specifically, we split into 8 2×2 additions/subtractions and 7 $2 \times 2 \times 2 \times 2$ multiplications
 $\approx 7 * 8 = 56$ multiplications

$$\downarrow$$

$$2^3 = 8$$

$$c) T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

7 subproblems + $O(n^2)$ additions/subtractions & recombinations

$$d) a = 7 \quad a > b^d$$

$$b = 2$$

$$d = 2$$

$$T(n) = n^{\log_2 7}$$

$$e) \text{Naive} \rightarrow O(n^3)$$

$$\text{Strassen} \rightarrow O(n^{2.81})$$

- Strassen asymptotically faster than naive ($2.81 < 3$)
- however Strassen has larger constants and overhead so for smaller matrices, naive is usually faster

QUESTION #04

$$\bullet \text{CASE 1: } a < b^d \rightarrow O(n^d)$$

non recursive work grows slower than n^d

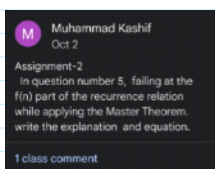
$$\bullet \text{CASE 2: } a = b^d \rightarrow O(n^d \log n)$$

both parts contribute equally (non recursive matches n^d)

$$\bullet \text{CASE 3: } a > b^d \rightarrow O(n^{\log_b a})$$

non recursive work dominates

QUESTION #05



- no, the master theorem cannot be used if the modified recurrence relation's $f(n)$ term is not polynomial.
- general recurrence eq $\rightarrow T(n) = aT\left(\frac{n}{b}\right) + f(n)$
- failure case example $\rightarrow f(n) = 2^n$ (exponential)
- reason for failure $\rightarrow$ it does not fit into one of the 3 established cases

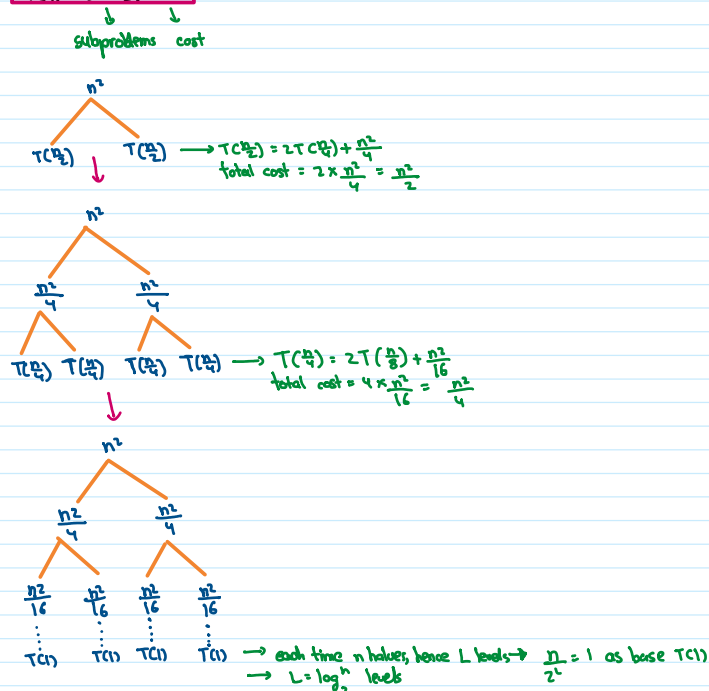
term is not polynomial.

- general recurrence eq $\rightarrow T(n) = aT(\frac{n}{b}) + f(n)$
- failure case example $\rightarrow f(n) = 2^n$ (exponential)
- reason for failure $\rightarrow$ it does not fit into one of the 3 established cases
- other failure cases $\rightarrow T(n)$ is not monotone ($T(n) = \sin n$)
 $\rightarrow$ cannot be expressed as a constant ($T(n) = T(\sqrt{n})$)

QUESTION #06

- 1) $T(n) = 2T(\frac{n}{2}) + n^2$
 using recurrence tree

$$T(n) = 2T(\frac{n}{2}) + n^2$$



TOTAL FULL COST:

$$\text{total cost at each level} = \frac{n^2}{2^i}$$

$$\text{total cost of all levels} = \frac{n^2}{2} + \frac{n^2}{4} + \frac{n^2}{8} + \dots + \frac{n^2}{2^{\log_2 n}} \quad \text{geometric series, hence sum converges to } \frac{a}{1-r}$$

$$= \frac{n^2}{1-\frac{1}{2}} = 2n^2 = \Theta(n^2)$$

per leaf node cost as each leaf base case $T(1) = c$ (a constant)

total leaves $\rightarrow 2^{\log_2 n} = n$

total leaf cost = $cn \rightarrow \Theta(n)$

$$T(n) = \Theta(n) + \Theta(n^2) = \Theta(n^2)$$

using iterative

$$T(n) = 2T(\frac{n}{2}) + n^2$$

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + \frac{n^2}{4}$$

substituting

$$T(n) = 2[2T(\frac{n}{4}) + \frac{n^2}{4}] + n^2 = 4T(\frac{n}{4}) + \frac{n^2}{2} + n^2 \quad \text{--- ①}$$

$$T(\frac{n}{4}) = 2T(\frac{n}{8}) + \frac{n^2}{16}$$

substituting into ①

$$T(n) = 4[2T(\frac{n}{8}) + \frac{n^2}{16}] + \frac{n^2}{2} + n^2 = 8T(\frac{n}{8}) + n^2 + \frac{n^2}{2} + \frac{n^2}{4}$$

pattern after k iterations: $\frac{n^2}{2^k}$

$$T(n) = 2^k T(\frac{n}{2^k}) + n^2 \sum_{i=0}^{k-1} \frac{1}{2^i}$$

base case $\rightarrow \frac{n}{2^k} = 1$

$$\log_2 n = k$$

substituting

$$T(n) = 2^{\log_2 n} T(\frac{n}{2^{\log_2 n}}) + n^2 \sum_{i=0}^{\log_2 n - 1} \frac{1}{2^i}$$

$$T(n) = n T(1) + n^2 \cdot O(1) = O(n^2)$$

ii) $T(n) = 3T(\frac{n}{3}) + n \log^2 n$
using iterative substitution

$$T(\frac{n}{3}) = 3T(\frac{n}{9}) + \frac{n}{3} \log^2 \frac{n}{3}$$

$$= 3T(\frac{n}{9}) + \frac{n}{3} \log^2 \frac{n}{3}$$

substituting

$$T(n) = 3[3T(\frac{n}{9}) + \frac{n}{3} \log^2 \frac{n}{3}] + n \log^2 n$$

$$= 9T(\frac{n}{9}) + n \log^2 \frac{n}{3} + n \log^2 n \quad \textcircled{1}$$

$$T(\frac{n}{9}) = 3T(\frac{n}{27}) + \frac{n}{9} \log^2 \frac{n}{9}$$

substituting

$$T(n) = 9[3T(\frac{n}{27}) + \frac{n}{9} \log^2 \frac{n}{9}] + n \log^2 \frac{n}{3} + n \log^2 n$$

$$= 27T(\frac{n}{27}) + n \log^2 \frac{n}{9} + n \log^2 \frac{n}{3} + n \log^2 n \quad \textcircled{2}$$

after k expansions

$$T(n) = 3^k T(\frac{n}{3^k}) + n \sum_{i=0}^{k-1} \log^2(\frac{n}{3^i})$$

base case $\rightarrow \frac{n}{3^k} = 1$

$$\log_3 n = k$$

substituting

$$T(n) = 3^{\log_3 n} T(\frac{n}{3^{\log_3 n}}) + n \sum_{i=0}^{\log_3 n - 1} \log^2(\frac{n}{3^i})$$

$$T(n) = n T(1) + \Theta(n \log^2 n)$$

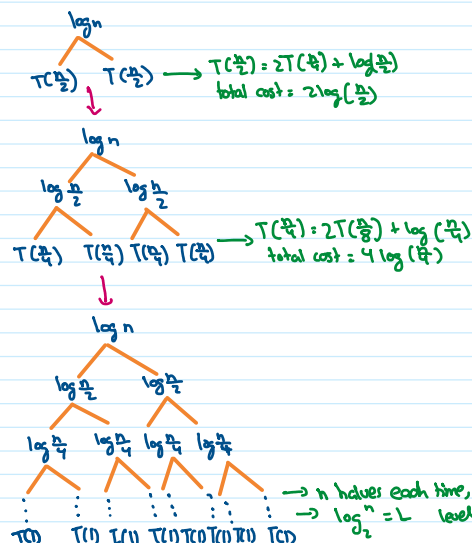
$$= \Theta(n) + \Theta(n \log^2 n)$$

$$= \Theta(n \log^2 n)$$

iii) $T(n) = 2T(\frac{n}{2}) + \log n$
using recurrence tree

$$T(n) = 2T(\frac{n}{2}) + \log n$$

subproblems cost



Full TOTAL COST

cost at each level: $2^i \log(\frac{n}{2^i})$

total cost for all levels: $\log n + 2 \log \frac{n}{2} + 4 \log \frac{n}{4} + 8 \log \frac{n}{8} + \dots + 2^{\log_2 n} \log \frac{n}{2^{\log_2 n}} \rightarrow$ geometric progression $\frac{a}{1-r}$

$$= \sum_{i=0}^{\log_2 n - 1} 2^i \log(\frac{n}{2^i}) \quad \log_2 n - 1 \text{ to separate leaf nodes}$$

$$= \sum_{i=0}^{\log_2 n - 1} 2^i (\log n - \log 2^i) \rightarrow \text{taking base 2}$$

$$= \sum_{i=0}^{\log_2 n - 1} 2^i (\log n - i)$$

$$\sum_{i=0}^{\log_2 n - 1} 2^i \log n \quad \sum_{i=0}^{\log_2 n - 1} 2^i i$$

$$= \log n \sum_{i=0}^{\log_2 n - 1} 2^i$$

geometric series sum

$$= 2^{\log_2 n} - 1$$

$$\text{largest term} = \log_2 n - 1$$

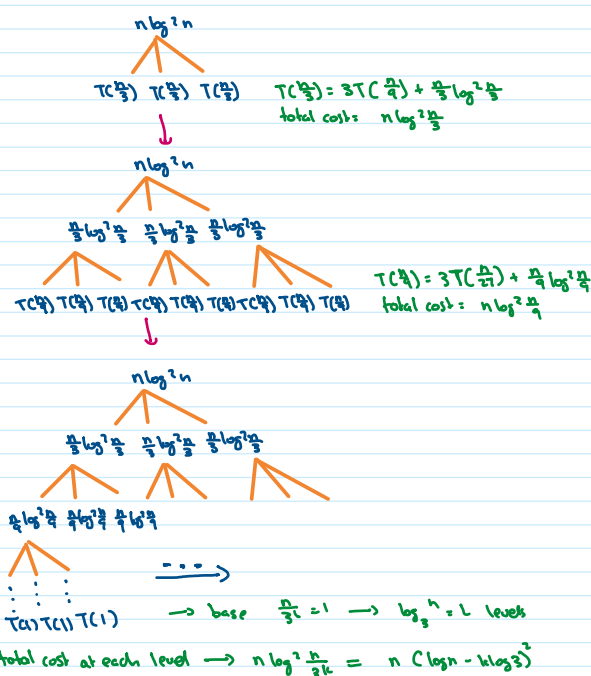
$$2^{\log_2 n} (\log_2 n) = n \log n$$

sum grows linearly with $n \rightarrow \Theta(n \log n)$

using recurrence tree

$$T(n) = 3T(\frac{n}{3}) + n \log^2 n$$

subproblems cost



total cost at each level $\rightarrow n \log^2 \frac{n}{3^k} = n (\log n - k \log 3)^2$

$$\rightarrow n \log^2 n, n \log^2 \frac{n}{3}, n \log^2 \frac{n}{9}, \dots, n \log^2 \frac{n}{3^{\log_3 n}}$$

time complexity $\rightarrow O(n \log^2 n)$

time complexity for leaves $\rightarrow O(n)$

$$T(n) = \Theta(n \log^2 n) + O(n)$$

$$T(n) = O(n \log^2 n)$$

using iterative substitution

$$T(n) = 2T(\frac{n}{2}) + \log n$$

$$T(\frac{n}{2}) = 2T(\frac{n}{4}) + \log \frac{n}{2}$$

substituting

$$T(n) = 2[2T(\frac{n}{4}) + \log \frac{n}{2}] + \log n$$

$$= 4T(\frac{n}{4}) + 2 \log \frac{n}{2} + \log n \quad \textcircled{1}$$

$$T(\frac{n}{4}) = 2T(\frac{n}{8}) + \log \frac{n}{4}$$

substituting into ①

$$T(n) = 4[2T(\frac{n}{8}) + \log \frac{n}{4}] + 2 \log \frac{n}{2} + \log n$$

$$= 8T(\frac{n}{8}) + \log n + 2 \log \frac{n}{2} + 4 \log \frac{n}{4}$$

after k iterations, pattern $\rightarrow$

$$T(n) = 2^k T(\frac{n}{2^k}) + \sum_{i=0}^{k-1} 2^i \log(\frac{n}{2^i})$$

base case $\rightarrow \frac{n}{2^k} = 1$

$$\log_2 n = k$$

substituting

$$T(n) = 2^{\log_2 n} T(\frac{n}{2^{\log_2 n}}) + \sum_{i=0}^{\log_2 n - 1} 2^i \log(\frac{n}{2^i})$$

$$T(n) = n T(1) + O(n)$$

$$T(n) = O(n)$$

$$\begin{aligned} & \text{geometric series sum} \\ & 2^k - 1 \\ & = 2^{\log_2 n} - 1 \\ & = n - 1 = \Theta(n) \\ & = \Theta(n \log n) \end{aligned}$$

$$2^{\log_2 n} = n \log n$$

sum grows linearly with $n \rightarrow \Theta(n \log n)$

$$\begin{aligned} \text{total cost of leaves} &= cn = \Theta(n) \\ T(n) &= O(n \log n) - O(n \log n) \\ T(n) &= O(n) \end{aligned}$$

QUESTION #07

$$1) T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$\begin{array}{ll} a=4 & a=b^d \\ b=2 & 4=2^2 \\ d=2 & 4=4 \end{array}$$

$$T(n) = \Theta(n^2 \log n)$$

$$2) T(n) = 2T\left(\frac{n}{4}\right) + \sqrt{n}$$

$$\begin{array}{ll} a=2 & a=b^d \\ b=4 & 2=4^{\frac{1}{2}} \\ d=\frac{1}{2} & 2=\sqrt{4} \\ & 2=2 \end{array}$$

$$T(n) = \Theta(\sqrt{n} \log n)$$

$$3) T(n) = 4T\left(\frac{n}{2}\right) + n^2 \log n$$

$$\begin{array}{ll} a=4 & a=b^d \\ b=2 & 4=2^2 \\ d=2 & 4=4 \end{array}$$

$$T(n) = \Theta(n^2 \log n)$$

QUESTION #08

$$1) T(n) = 4T\left(\frac{n}{2}\right) + n^2$$

$$\text{guess 1} \rightarrow T(n) = O(n) \quad \text{no}$$

$$\begin{aligned} T(n) &\leq cgn \quad \text{for } c > 0 \\ 4T\left(\frac{n}{2}\right) + n^2 &\leq cn \end{aligned}$$

$$4 \cdot \frac{cn}{2} + n^2 \leq cn$$

$$2cn + n^2 \leq cn \quad \text{no wrong}$$

$$\text{guess 2} \rightarrow T(n) = O(n^2)$$

$$\begin{aligned} T(n) &\leq cn^2 \quad \text{for } c > 0 \\ 4T\left(\frac{n}{2}\right) + n^2 &\leq cn^2 \end{aligned}$$

$$4 \cdot \frac{cn^2}{4} + n^2 \leq cn^2$$

$$cn^2 + n^2 \leq cn^2$$

$$n^2(c+1) \leq cn^2 \quad c+1 > c$$

no wrong